

Objective

Upon completing this tutorial, you should understand:

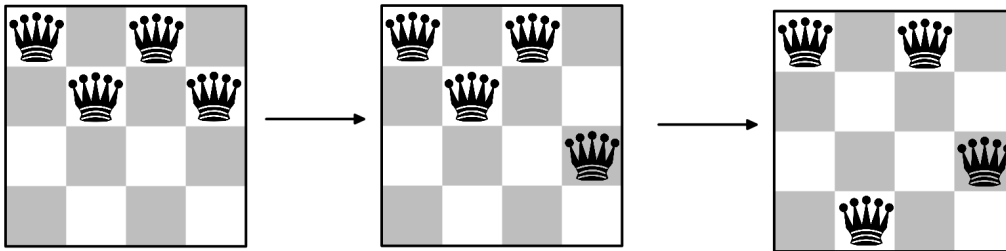
- Be familiar with the maximum flow and stable marriage problems.

Iterative improvement

Iterative improvement refers to starting with some feasible solution (that is a solution that at least satisfies all constraints of the problem) and proceeds to improve it by repeated applications of some simple step. For iterative improvement algorithms to work which are:

- **Initial feasible solution:** We can start with some trivial or approximate solution obtained by another algorithm. Finding an initial solution may require a lot of effort depending on the type of problem.
- **Identify allowed changes in feasible solution:** It may not be obvious what allowed changes should be allowed to the initial feasible solution so that we can determine if the change is locally optimal.
- **Local versus global extremum:** We need to ensure that our solutions aren't just the best in a "local" instance but the best globally.

Consider the example of a chess board of size $n \times n$ where no two queens can be on the same row, column or diagonal. Suppose we start with a board with several conflicts. On each iteration, we reduce the number of conflicts until we've found the best solution.



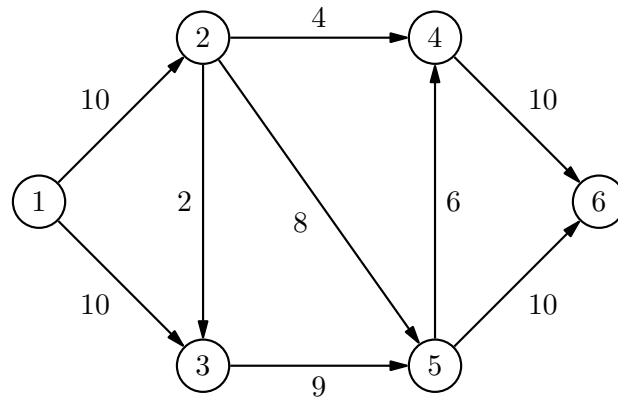
The maximum-flow problem

Lets consider the problem of maximizing the flow of water through a transportation network. We will represent this as a connected weighted graph with n vertices numbered from 1 to n and a set of edges E , with the following properties:

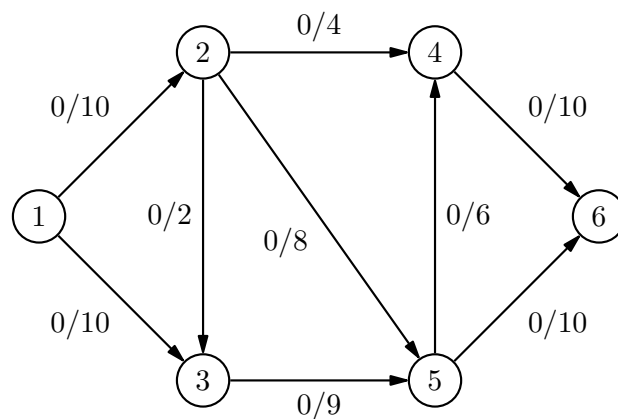
- It contains exactly one vertex with no entering edges which is called the **source** and assumed to be numbered 1.
- It contains exactly one vertex with no leaving edges which is called the **sink** and assumed to be numbered n .
- The weight w_{ft} of each directed edge (f, t) is a positive integer called the **capacity** and this represents the maximum amount of flow that can be sent from f to t through the edge.

The algorithm we are going to use to find the maximum-flow is called the ***Ford-Fulkerson method*** in which in each iteration of the algorithm, we generate a path from the source node to the sink node in which some additional flow can be sent. Note that this algorithm is actually fairly intricate underneath the hood but the example will show how it works.

Lets consider the following example in which we want to work out the maximum flow for:



We want to find the maximum amount of total flow that can be delivered from node 1 to node 6. Each weighting represents the maximum amount of flow that can be transported through the edge at any time - you can think of these directed edges as a pipe. Lets represent the edges now as *current flow/maximum capacity* where the *maximum capacity* is the weight of the edge.

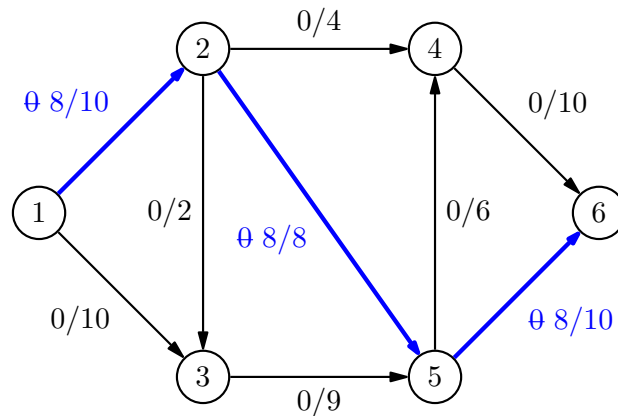


Total flow: 0

As stated, on each iteration of the algorithm we find a path from the source node (1) to the sink node (6) in which additional flow can be sent; such a path is called ***flow augmenting***. When we find such paths, we don't just find paths where there is a forward edge, that is an edge from f to t , but we consider backwards edges from t to f as well. So we can interpret this as finding paths in the underlying *undirected* graph to maximize the flow; the reason for this is to maximize the amount of flow on which a previous iteration may not have done so (this will become clearer later).

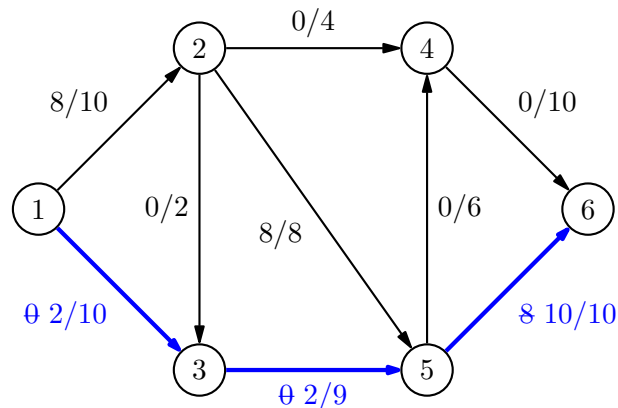
CONTINUE TO NEXT PAGE

First, let's pick an augmented path from the source to the sink. Let's consider the path $1 \rightarrow 2 \rightarrow 5 \rightarrow 6$. The maximum amount of flow that can be sent is 8 since this is bottlenecked from $2 \rightarrow 5$. Whatever flow we put in for a node we must also be able to put out. Let's update the graph as follows



Total flow: 8

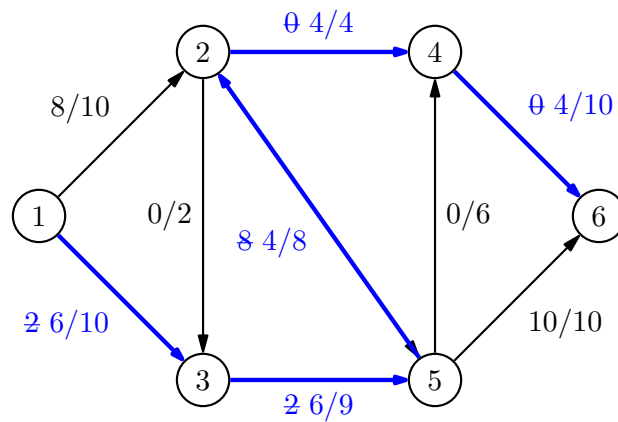
The total flow is now 8. Considering another augmented path, let's consider $1 \rightarrow 3 \rightarrow 5 \rightarrow 6$. We know the flow from $5 \rightarrow 6$ is 8 and the maximum capacity is 10, so the maximum we can send in this augmented path is 2. Let's update the path as follows.



Total flow: 10

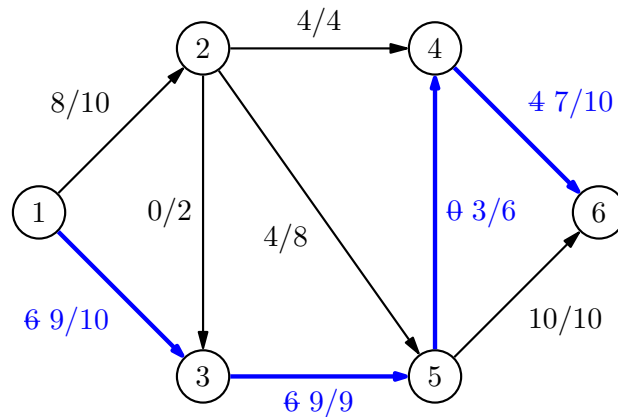
Let's consider the augmented path of $1 \rightarrow 3 \rightarrow 5 \leftarrow 2 \rightarrow 4 \rightarrow 6$. Observe that this path has a backward edge. Firstly, the bottleneck capacity of this path is 4 from the path $2 \rightarrow 4$. How this works, is we add 4 to the current flow for each forward edge, but minus 4 from the backward edge of $5 \leftarrow 2$. The reason for considering backward edges is to "improve" a previous iteration of this algorithm for a better one; if we could not perform this step, then we may not actually find the maximum flow if we picked an "unfortunate" set of augmented paths.

CONTINUE TO NEXT PAGE



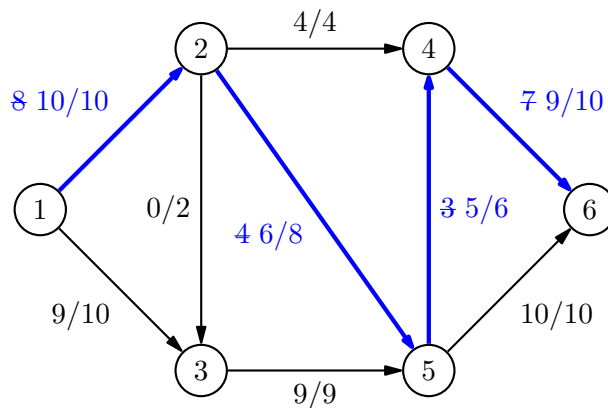
Total flow: 14

Now we'll consider $1 \rightarrow 3 \rightarrow 5 \rightarrow 4 \rightarrow 6$. The maximum flow we can add to here is 3 since from $3 \rightarrow 5$ the current flow is 6/9.



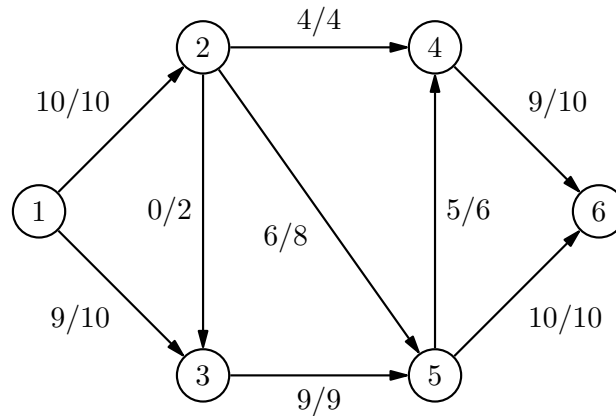
Total flow: 17

Now consider the augmented path of $1 \rightarrow 2 \rightarrow 5 \rightarrow 4 \rightarrow 6$. The maximum flow that can be delivered here is 2 since $1 \rightarrow 2$ is currently at 8/10.



Total flow: 19

Observing the graph, $1 \rightarrow 2$ has maximum flow delivered at 10/10. $1 \rightarrow 3$ has 1 flow available but $3 \rightarrow 5$ is full and the backward edge from $3 \leftarrow 2$ is currently at 0 so we can't subtract any flow from here. Since we can't find any more augmented paths, the algorithm halts and we have reached the maximum flow at 19.

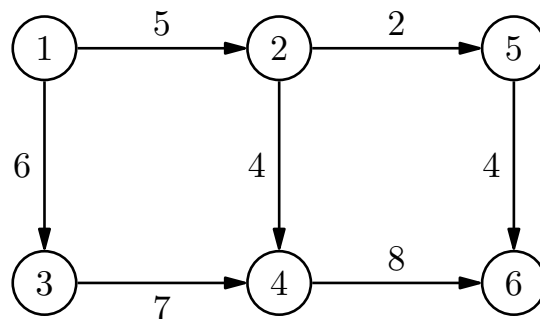


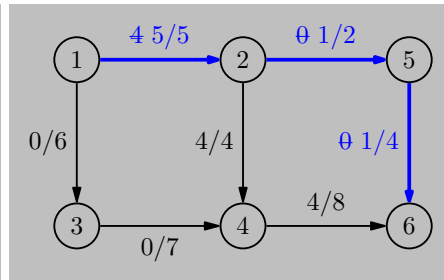
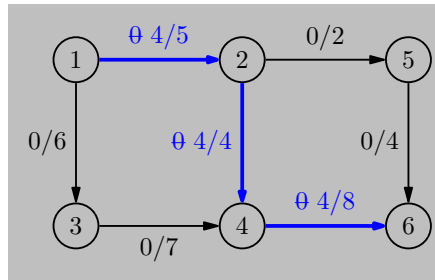
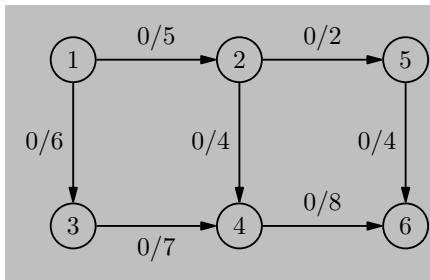
Final maximum flow: 19

It's important to note that the augmented paths generated wasn't the most optimal and a poor generation of augmented paths can impact the algorithm's performance. In fact the algorithm itself does not specify a way to generate augmented paths; one way of generating augmented paths is by using breadth-first search to generate paths with the least number of edges - more can be read online on how paths are generated and as mentioned, this algorithm is fairly intricate.

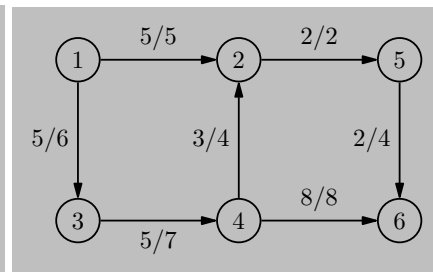
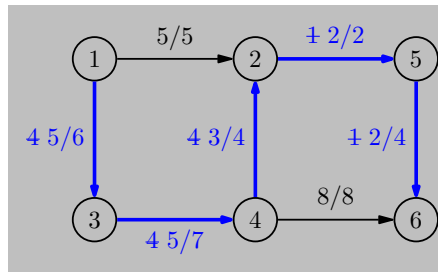
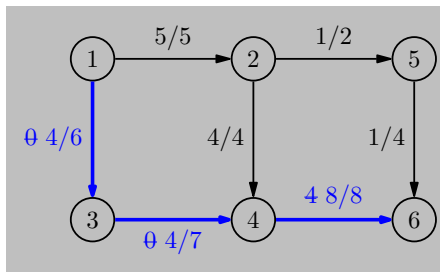
Questions

1. Apply the shortest-path Ford Fulkerson algorithm to find a maximum flow in the following networks. The source is vertex 1, and the sink is vertex 6.





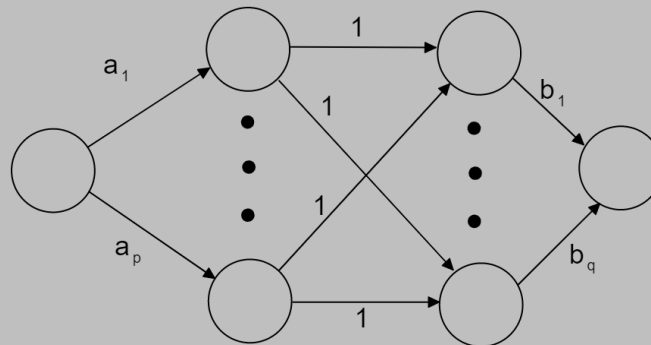
Total flows of 0, 4 and 5



Total flows of 9 with the max flow of 10 being delivered.

- Several families go out to dinner together. To increase their social interaction, they would like to sit at tables so that no two members of the same family are at the same table. Show how to find a seating arrangement that meets this objective (or prove that no such arrangement exists) by using a maximum-flow problem. Assume that the dinner contingent has p families and that the i th family has a_i members. Also assume that q tables are available and the j th table has a seating capacity of b_j .

Solve the maximum flow problem for the following network:



If the maximum indicated flow by the value full is edges equal of to capacity $\sum_{i=1}^p a_i$ then in the the maximum problem has flow; otherwise, the problem does not have a solution.

3. Find a stable matching for the instance defined by the following ranking matrices and group 1 is proposing:

Preferences	A	B	C	D
1st	δ	β	δ	γ
2nd	γ	δ	β	β
3rd	α	α	γ	α
4th	β	γ	α	δ

Table 1: Group 1 preferences.

Preferences	α	β	γ	δ
1st	A	A	B	D
2nd	B	D	A	B
3rd	C	C	C	C
4th	D	B	D	A

Table 2: Group 2 preferences.

Round 1: A propose to δ , B to β , C to δ and D to γ .

Tentative match: B to *beta*, C to δ , D to γ

Round 2: A proposes to γ

Tentative match: A to γ , B to β , C to δ

Round 3: D proposes to β

Tentative match: A to γ , C to δ , D to β

Round 4: B proposes to δ

Tentative match: A to γ , B to δ , D to β

Round 5: C proposes to β

Tentative match: A to γ , B to δ , D to β

Round 6: C proposes to γ

Tentative match: A to γ , B to δ , D to β

Round 5: C proposes to α

Final match: A to γ , B to δ , C to α , D to β

4. Consider the following the post office location problem: Given n integers $x_1, x_2, \dots, x_n$ representing coordinates of n villages located along a straight road, find a location for a post office that minimizes the average distance between the villages. The post office may be, but is not required to be, located at one of the villages. Devise an iterative improvement algorithm for this problem. Is this an efficient way to solve this problem?

The problem is to find a value x that minimizes $f(x) = \frac{1}{n} \sum_{i=1}^n |x - x_i|$, which is the total sum of all the distances from the villages to the post office which is multiplied by $\frac{1}{n}$ as it's the average.

This can be simplified to $g(x) = \sum_{i=1}^n |x - x_i|$ as $\frac{1}{n}$ is constant.

Here is an iterative improvement algorithm. Pick an arbitrary integer x and compute $g(x)$. Then compute $g(x - 1)$ and $g(x + 1)$. If $g(x - 1) < g(x)$ or $g(x + 1) < g(x)$ then replace x by $x - 1$ or $x + 1$ respectively as this implies the new location has a smaller average than the previous x ; repeat this step with the new value of x otherwise return x as the post office location.